Learning Module: W5M1

March 11, 2026

Contents

1 1. Introduction & Motivation **4**

2 Parametric versus Non-Parametric Models **4**
2.1 Definitions . . . 4
2.2 Flexibility and Approximation Capability . . . 4
2.3 Bias–Variance Trade-off . . . 4
2.4 Curse of Dimensionality . . . 5
2.5 Practical Implications . . . 5
2.6 Summary . . . 5
2.7 Questions . . . 6

3 2. Description of k-NN Regression **7**
3.1 Theory . . . 7
3.2 From Parametric to Non-Parametric Learning . . . 7
3.3 Problem Setting . . . 8
3.4 Algorithmic Procedure . . . 8
3.5 Key Characteristics . . . 9
3.6 Bias–Variance Trade-off and Choice of k . . . 9
3.7 Asymptotic Consistency . . . 9
3.8 Computational Considerations . . . 9
3.9 Practical Tips . . . 10
3.10 Summary . . . 10
3.11 Questions . . . 10

4 3. Algorithm Steps **12**
4.1 Algorithmic Procedure for k-Nearest Neighbours Regression . . . 12
4.1.1 Notation . . . 13
4.1.2 Step 1 – Distance Computation . . . 13
4.1.3 Step 2 – Selection of the k Nearest Neighbors . . . 13
4.1.4 Step 3 – Prediction by Averaging . . . 14
4.1.5 Algorithmic Summary . . . 14
4.1.6 Connection to the Non-Parametric Paradigm . . . 14
4.1.7 Practical Remarks . . . 14

5 4. Advantages **17**
5.1 Desirable Properties of the k-Nearest Neighbours Regressor . . . 17
5.2 Simplicity . . . 17
5.3 Versatility . . . 18
5.4 Interpretability . . . 18
5.5 Flexibility . . . 18
5.6 Summary . . . 19

6 5. Limitations **21**
6.1 Disadvantages of k-Nearest Neighbours Regression . . . 21
6.1.1 Bias–Variance Trade-off Governed by k . . . 21
6.1.2 Sensitivity to Outliers and Noise . . . 21
6.1.3 Computational Burden at Prediction Time . . . 21

6.1.4 Curse of Dimensionality . . . 22
6.1.5 Impact of Region Imbalance . . . 22
6.1.6 Summary . . . 22

7 Worked Example **24**
7.1 Theory . . . 24
7.2 Questions . . . 25

8 7. Summary **27**
8.1 Parametric vs. Non-Parametric Learning . . . 27
8.1.1 Parametric models . . . 27
8.1.2 Non-parametric models . . . 27
8.1.3 Bias–Variance intuition . . . 27
8.2 Key Practical Considerations When Building Machine-Learning Models . 27
8.3 k-Nearest Neighbour Regression . . . 28
8.3.1 Definition . . . 28
8.3.2 Weighted variants . . . 29
8.3.3 Algorithmic steps . . . 29
8.3.4 Worked example (univariate) . . . 29
8.4 Questions . . . 29

1 1. Introduction & Motivation

2 Parametric versus Non-Parametric Models

2.1 Definitions

A **parametric model** is characterised by a fixed, finite set of parameters $\boldsymbol{\theta} \in \mathbb{R}^p$ that completely determine the functional form of the predictor. Once the parameters have been estimated from data, the model no longer depends on the training set. Linear regression is the canonical example:

$$y = \underbrace{\beta_0 + \beta_1 x_1 + \cdots + \beta_d x_d}_{\text{linear predictor}} + \varepsilon, \qquad \boldsymbol{\beta} = (\beta_0, \beta_1, \ldots, \beta_d)^\top,$$

where $p = d + 1$ is the number of coefficients to be learned.

A **non-parametric model** does not assume a fixed-size parameter vector. Instead, the model complexity can grow with the amount of training data, allowing the predictor to adapt to arbitrarily rich functional relationships. The k-nearest neighbours (k-NN) algorithm exemplifies this class:

$$\hat{y}(\mathbf{x}) = \begin{cases} \dfrac{1}{k} \displaystyle\sum_{i \in \mathcal{N}_k(\mathbf{x})} y_i, & \text{(regression)}, \\ \arg\max_c \displaystyle\sum_{i \in \mathcal{N}_k(\mathbf{x})} \mathbf{1}\{y_i = c\}, & \text{(classification)}, \end{cases}$$

where $\mathcal{N}_k(\mathbf{x})$ denotes the index set of the k training points whose covariates are closest to $\mathbf{x}$ according to a chosen distance metric (typically Euclidean).

2.2 Flexibility and Approximation Capability

The flexibility of a learning method can be formally related to the size of its *hypothesis space* $\mathcal{H}$:

$$\mathcal{H}_{\text{parametric}} = \{f_{\boldsymbol{\theta}} : \boldsymbol{\theta} \in \mathbb{R}^p\}, \qquad \mathcal{H}_{\text{non-parametric}} = \bigcup_{n=1}^{\infty} \mathcal{H}_n,$$

where $\mathcal{H}_n$ is the set of functions that can be represented using n training examples. Because $\mathcal{H}_{\text{non-parametric}}$ expands as more data become available, non-parametric learners can approximate highly non-linear, high-dimensional relationships that would require an impractically large number of parameters in a parametric model.

2.3 Bias–Variance Trade-off

The statistical risk R of an estimator $\hat{f}$ can be decomposed into bias and variance components:

$$\mathbb{E}\Big[(Y - \hat{f}(\mathbf{X}))^2\Big] = \underbrace{\Big(\mathbb{E}[\hat{f}(\mathbf{X})] - f(\mathbf{X})\Big)^2}_{\text{Bias}^2} + \underbrace{\mathbb{E}\Big[(\hat{f}(\mathbf{X}) - \mathbb{E}[\hat{f}(\mathbf{X})])^2\Big]}_{\text{Variance}} + \sigma_\varepsilon^2,$$

where f is the true regression function and $\sigma_\varepsilon^2 = \operatorname{Var}(\varepsilon)$ is the irreducible noise.

- **Parametric models** (e.g., linear regression) tend to have low variance because they are constrained to a low-dimensional $\mathcal{H}$; however, if the true relationship is strongly non-linear, their bias can be large.
- **Non-parametric models** (e.g., k-NN) typically exhibit low bias—provided a sufficient amount of data—since $\mathcal{H}$ can approximate any continuous function arbitrarily well. Their variance, however, grows with the flexibility of the method (e.g., with decreasing k or with high dimensionality).

2.4 Curse of Dimensionality

For k-NN the distance between points becomes less discriminative as the dimensionality d increases. Formally, if $\mathbf{X}$ is uniformly distributed in $[0,1]^d$, the expected distance to the nearest neighbour scales as

$$\mathbb{E}[\|\mathbf{X}_{(1)} - \mathbf{X}\|] \approx \left(\frac{\log n}{n}\right)^{1/d},$$

where n is the training size. Consequently, achieving a fixed level of approximation error requires an exponential increase in n with d. This phenomenon explains why, despite their theoretical flexibility, non-parametric methods can deteriorate in very high dimensions unless additional structure (e.g., metric learning, dimensionality reduction) is imposed.

2.5 Practical Implications

1. **Model selection:** When the underlying relationship is believed to be close to linear and the data set is modest in size, a parametric model such as linear regression yields a parsimonious, interpretable solution with low variance.
2. **When to prefer non-parametric methods:** If exploratory analysis indicates strong non-linearity, interactions, or complex manifold structure, k-NN (or other non-parametric learners) can capture these patterns provided enough training examples are available.
3. **Hybrid strategies:** It is common to use a parametric model as a baseline and then augment it with non-parametric components (e.g., additive models, kernel smoothing) to achieve a favourable bias–variance balance.

2.6 Summary

Linear regression exemplifies a parametric approach: a finite set of coefficients fully specifies the predictor, leading to low variance but potentially high bias for non-linear phenomena. In contrast, the k-nearest neighbours algorithm is non-parametric; its hypothesis space grows with the data, granting it the capacity to model complex, high-dimensional, non-linear relationships at the cost of increased variance and sensitivity to the curse of dimensionality. Understanding this dichotomy is essential for selecting the appropriate modelling strategy for a given regression or classification task.

2.7 Questions

1. **K-NN regression calculation (easy).**
   Given the training set $\{(2,4),\ (5,8),\ (8,12),\ (11,16)\}$, predict the response for a query point $x = 7$ using $k = 3$. Show all steps, including the identification of the three nearest neighbours and the averaging of their target values.

2. **Expected nearest-neighbour distance (easy–medium).**
   Assume covariates are uniformly distributed in the unit hypercube $[0,1]^d$. Using the approximation
   $$\mathbb{E}[\|\mathbf{X}_{(1)} - \mathbf{X}\|] \approx \left(\frac{\log n}{n}\right)^{1/d},$$
   compute the expected distance for (a) $n = 5\,000,\ d = 2$ and (b) $n = 5\,000,\ d = 10$. Comment on what the numbers imply for the usefulness of k-NN in low versus high dimensions.

3. **Bias–variance intuition for parametric models (medium).**
   Explain why a parametric model such as linear regression can exhibit low variance but high bias when the true regression function is highly non-linear. In your answer refer explicitly to the size of the hypothesis space $\mathcal{H}_{\text{parametric}}$ and the role of the fixed parameter vector $\boldsymbol{\theta}$.

4. **Bias–variance behavior of k-NN as k varies (medium).**
   Describe qualitatively how the bias and variance of a k-NN regressor change when we move from $k = 1$ (the nearest-neighbour rule) to $k = n$ (predicting the overall mean). Relate your discussion to the bias–variance decomposition presented in the theory.

5. **When linear regression is preferable (medium).**
   List three practical scenarios in which a linear regression model would be chosen over k-NN even if the true relationship is mildly non-linear. For each scenario, identify which aspect of the bias–variance trade-off or which practical consideration (e.g., interpretability, computational cost) dominates the decision.

6. **Designing a modelling strategy for large, high-dimensional data (hard).**
   You have a data set with $n = 10\,000$ observations and $d = 25$ features, and exploratory analysis suggests a strong non-linear relationship between the response and the predictors. Propose a modelling pipeline that balances bias and variance. State whether you would begin with a parametric model, a non-parametric model, or a hybrid approach, and describe any preprocessing (e.g., feature selection, dimensionality reduction) you would perform to mitigate the curse of dimensionality.

7. **Monte-Carlo study of bias–variance trade-off (hard).**
   Design a simulation experiment to empirically illustrate the bias–variance trade-off for linear regression versus k-NN as a function of sample size n and dimensionality d. Specify:

   (a) a data-generating process with a known non-linear true function f^*;

   (b) the values of k you would examine;

(c) how you would estimate bias, variance, and total risk from the simulated replications;

(d) a visualisation strategy (e.g., heat-maps or bias/variance curves) that makes the impact of the curse of dimensionality evident.

8. **Hybrid additive model (hard).**
Outline a pseudo-algorithm for fitting an additive model that combines a linear term with a k-NN smoother applied to the residuals. Explain how this two-stage procedure can achieve a more favourable bias–variance balance than either component alone.

9. **Metric learning for high-dimensional k-NN (hard).**
Suppose only 5 out of 20 features are truly relevant for the prediction task. Propose a strategy to learn an appropriate distance metric or to preprocess the data (e.g., via feature weighting or selection) so that irrelevant dimensions do not degrade neighbour search. Discuss the connection of this strategy to the curse of dimensionality.

10. **Kernel-weighted k-NN (hard).**
Consider replacing the hard Euclidean neighbourhood in k-NN with a Gaussian kernel weight $K(\mathbf{x}, \mathbf{x}_i) = \exp\Big(- \|\mathbf{x} - \mathbf{x}_i\|^2 / h^2 \Big)$. Discuss how this modification changes the hypothesis space $\mathcal{H}_{\text{non-parametric}}$ and the bias–variance characteristics of the estimator. Under what conditions (in terms of bandwidth h and data density) might the kernel-weighted version outperform the standard nearest-neighbour rule?

11. **Critical appraisal of a real-world k-NN application (hard).**
A study applied k-NN regression to predict house prices using 100 features (location, size, age, and many dummy-encoded categorical variables). The reported test R^2 was lower than that of a simple linear regression using only three features (size, age, distance to city centre). Identify at least three theoretical reasons (drawn from the sections on bias–variance, curse of dimensionality, metric choice, etc.) that could explain this outcome. Propose concrete steps to improve the non-parametric model's performance (e.g., feature engineering, metric learning, dimensionality reduction, tuning k, etc.).

3 2. Description of k-NN Regression

3.1 Theory

3.2 From Parametric to Non-Parametric Learning

In the preceding sections we distinguished two broad families of learning algorithms:

- **Parametric models** assume a fixed-size set of parameters (e.g., the coefficients of a linear model). Their functional form is specified a priori, and learning consists of estimating those parameters from data.

- **Non-parametric models** place no such a priori restriction on the number of parameters. Instead, the data themselves determine the model's complexity; as more training points become available the effective model can become richer.

k-Nearest Neighbours (k-NN) exemplifies the non-parametric paradigm. It does not fit an explicit functional form; it simply stores the training set and lets the geometry of the data dictate predictions. Because it is also a *lazy* learner—deferring all computation until a query is made—it offers a clean illustration of the trade-offs inherent to non-parametric methods.

3.3 Problem Setting

Let a training set of n examples be given as

$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{n}, \qquad \mathbf{x}_i \in \mathbb{R}^d,\ y_i \in \mathbb{R},$$

where $\mathbf{x}_i$ denotes the feature vector and y_i the associated continuous target (regression) value. For a new query point $\mathbf{x}_q$ we wish to predict its target $\hat{y}(\mathbf{x}_q)$.

3.4 Algorithmic Procedure

1. **Distance computation:** Compute a distance (or similarity) measure

   $$d(\mathbf{x}_q, \mathbf{x}_i) \quad \text{for all } i = 1, \ldots, n.$$

   The Euclidean distance $d(\mathbf{x}_q, \mathbf{x}_i) = \|\mathbf{x}_q - \mathbf{x}_i\|_2$ is most common, but other metrics (Manhattan, Mahalanobis, cosine, ...) may be used depending on the data characteristics.

2. **Neighbour selection:** Identify the index set

   $$\mathcal{N}_k(\mathbf{x}_q) = \{i_1, \ldots, i_k\},$$

   where the k indices correspond to the training samples with the smallest distances to $\mathbf{x}_q$.

3. **Prediction:** Average (or weighted-average) the target values of the selected neighbours:

   $$\boxed{\hat{y}(\mathbf{x}_q) = \frac{1}{k} \sum_{i \in \mathcal{N}_k(\mathbf{x}_q)} y_i} \tag{1}$$

   or, with distance-based weights $w_i = w\Big(d(\mathbf{x}_q, \mathbf{x}_i)\Big)$:

   $$\boxed{\hat{y}(\mathbf{x}_q) = \frac{\displaystyle\sum_{i \in \mathcal{N}_k(\mathbf{x}_q)} w_i\, y_i}{\displaystyle\sum_{i \in \mathcal{N}_k(\mathbf{x}_q)} w_i}} \tag{2}$$

   Common choices for w_i include inverse distance $w_i = 1/d(\mathbf{x}_q, \mathbf{x}_i)$ or kernel functions such as the Gaussian $w_i = \exp\Big(-d^2/2\sigma^2\Big)$.

3.5 Key Characteristics

Non-parametric No explicit parameters are learned; the effective model complexity grows with the size of $\mathcal{D}$.

Lazy learning The training phase consists solely of storing $\mathcal{D}$. All computational effort (distance calculations, neighbour search, averaging) occurs at prediction time.

Locality Predictions depend only on the *local* neighbourhood of the query point, embodying the intuition that "similar inputs have similar outputs."

3.6 Bias–Variance Trade-off and Choice of k

The hyper-parameter k governs the balance between bias and variance:

- **Small** k (e.g., $k = 1$) yields low bias because the prediction follows the nearest training point almost exactly. However, it is highly sensitive to noise, leading to large variance.
- **Large** k smooths over many points, reducing variance at the expense of higher bias (the prediction may overshoot local structure).

A common heuristic is to select k via cross-validation, seeking the value that minimizes an estimate of the generalisation error (e.g., mean-squared error on held-out folds).

3.7 Asymptotic Consistency

Under mild regularity conditions (continuous target function, a suitable distance metric, and a density that does not vanish), the k-NN regressor is *consistent*. Specifically, if the number of neighbours grows with the sample size while remaining a vanishing fraction:

$$k = k_n \quad \text{s.t.} \quad k_n \to \infty, \qquad \frac{k_n}{n} \to 0 \ \text{ as } n \to \infty,$$

then

$$\hat{y}(\mathbf{x}) \xrightarrow{P} \mathbb{E}[Y \mid \mathbf{X} = \mathbf{x}] \quad \text{for any fixed } \mathbf{x},$$

i.e., the estimator converges in probability to the true regression function.

3.8 Computational Considerations

A naïve implementation requires $O(nd)$ distance evaluations per query, which becomes prohibitive for large n or high dimensionality. Several strategies alleviate this burden:

- **Space-partitioning trees** (KD-trees, ball trees) reduce the average query cost to $O(\log n)$ in low-to-moderate dimensions.
- **Approximate nearest-neighbour** algorithms (e.g., locality-sensitive hashing) trade a small loss in neighbour accuracy for substantial speed gains.
- **Dimensionality reduction** (PCA, random projections) mitigates the "curse of dimensionality," which otherwise dilutes the notion of proximity because distances concentrate.

3.9 Practical Tips

1. **Feature scaling** is essential: Euclidean distance is sensitive to the units of each feature; standardising or normalising each dimension ensures that no single attribute dominates the distance computation.

2. **Distance metric selection** should reflect domain knowledge; for instance, Mahalanobis distance incorporates feature covariances and can improve performance when features are correlated.

3. **Weighting schemes** often improve accuracy, especially when the data contain outliers; inverse-distance weighting diminishes the influence of distant, potentially noisy neighbours.

3.10 Summary

k-Nearest Neighbours regression epitomises a simple yet powerful non-parametric, lazy-learning approach. By averaging the outputs of the k most similar training cases, it leverages the fundamental assumption that nearby points in feature space share similar target values. Its flexibility comes with trade-offs—chiefly the selection of k, the choice of distance metric, and computational scalability—but these are amenable to systematic tuning and algorithmic acceleration. Consequently, k-NN serves both as a pragmatic baseline for regression tasks and as a conceptual bridge between intuitive similarity-based reasoning and formal statistical learning theory.

3.11 Questions

1. **Definition and Basic Prediction (Easy)**
   Given the training set $\{(2,4),\ (5,8),\ (8,12),\ (11,16)\}$, predict the target for a query point with $x = 7$ using $k = 3$ and unweighted averaging. Show all steps.

2. **Parametric vs. Non-Parametric (Easy)**

   Explain why a non-parametric method such as k-NN can be advantageous when the underlying functional form is unknown.

   Provide a concrete scenario in which a parametric model (e.g., linear regression) would be preferred over k-NN, despite the latter's flexibility.

3. **Distance Metric Selection (Medium)**
   Two features, *height* (cm) and *age* (years), are highly correlated.

   Discuss the suitability of Euclidean distance for this data.

   Derive the Mahalanobis distance for a two-dimensional vector $\mathbf{x}$ given a covariance matrix Σ and explain how it compensates for correlation.

   Propose an empirical procedure (including validation protocol) to decide which metric—Euclidean, Manhattan, or Mahalanobis—yields the best k-NN performance on this dataset.

4. **Effect of Feature Scaling (Medium)**
   Features: salary ($30k–$120k), years of experience (0–30), binary certification (0/1).

Quantitatively illustrate how omitting scaling can distort Euclidean distances.

Standardise each feature to zero mean and unit variance (means: 70k, 15, 0.5; s.d.: 20k, 10, 0.5). Compute the Euclidean distance between $\mathbf{x}_a = (80\text{k}, 20, 1)$ and $\mathbf{x}_b = (50\text{k}, 5, 0)$ before and after standardisation.

Discuss the impact of this scaling on k-NN regression predictions.

5. **Bias–Variance Trade-off (Medium)**
For a regression task you obtain the following MSEs:

k	Training MSE	Validation MSE
1	0.02	1.30
50	0.90	0.45

Identify which k value is over-fitting and which is under-fitting; give a bias-variance explanation.

Sketch (conceptually) the expected validation-error-vs-k curve and label the high-bias, high-variance, and optimal regions.

Outline how you would use 5-fold cross-validation to select the optimal k, describing the steps and the decision criterion.

6. **Weighted vs. Unweighted Averaging (Medium)**
A query point $\mathbf{x}_q$ has three nearest neighbours with distances $d_1 = 0.5$, $d_2 = 1.0$, $d_3 = 1.5$ and target values $y_1 = 3$, $y_2 = 5$, $y_3 = 7$.

Compute the unweighted prediction ($k = 3$).

Compute the weighted prediction using inverse-distance weights $w_i = 1/d_i$.

Discuss how inverse-distance weighting can reduce the influence of outliers and under what conditions weighting might degrade performance.

7. **Asymptotic Consistency (Medium)**
The consistency condition requires $k_n \to \infty$ and $k_n/n \to 0$ as $n \to \infty$.

Give an explicit sequence k_n satisfying these conditions for $n = 10,\ 100,\ 1000,\ 10\,000$.

Explain intuitively why both conditions are needed for $\hat{y}(\mathbf{x})$ to converge to $\mathbb{E}[Y \mid \mathbf{X} = \mathbf{x}]$.

If the true regression function is discontinuous (e.g., a step function), discuss how this affects the consistency result.

8. **Curse of Dimensionality (Hard)**
Consider a unit hyper-cube $[0, 1]^d$ and a query point at its centre. Let r be the radius of a hypersphere that encloses a fixed proportion p (e.g., $p = 0.01$) of uniformly distributed training points.

Derive the relationship between r, p, and the dimension d.

Using the derived formula, compute r for $d = 2, 5, 10, 20$ with $p = 0.01$ and comment on the trend.

Relate this phenomenon to practical performance of k-NN regression in high dimensions and propose two mitigation strategies.

9. **Computational Strategies for Large-Scale Deployment (Hard)**
You must serve k-NN regression on a dataset with $n = 1\,000\,000$ points in $d = 15$ dimensions, answering queries in under 50ms on average.

   Estimate the number of distance computations required by a naïve linear scan per query and argue whether the latency target is realistic.

   Explain how a KD-tree can reduce average query cost, and specify the dimensional regime where KD-trees become ineffective.

   Design a hybrid solution that combines dimensionality reduction (e.g., PCA) with an approximate nearest-neighbour method (e.g., LSH). Outline the pipeline and justify how each component helps meet the latency requirement.

10. **Practical Evaluation and Model Selection (Hard)**
Three k-NN variants have been evaluated on the same validation set:

    - (A) Euclidean distance, $k = 7$, unweighted (MSE=0.84).
    - (B) Manhattan distance, $k = 7$, inverse-distance weighting (MSE=0.78).
    - (C) Mahalanobis distance (empirical covariance), $k = 12$, unweighted (MSE=0.76).

   Rank the three models by expected generalisation performance and discuss any trade-offs.

   Assuming strong feature correlation and a few noisy outliers, indicate which model you expect to be most robust and why.

   Propose a systematic experiment (including ranges for k, distance metrics, weighting schemes, and evaluation metrics) to test whether increasing k beyond 12 would further improve or deteriorate performance.

11. **Design Challenge: House-Price Prediction (Hard)**
You are to build a k-NN regression system for predicting house prices using the features: square footage, number of bedrooms, house age, distance to city centre, and neighbourhood crime rate.

    a) Propose a complete preprocessing pipeline (scaling, encoding, possible feature engineering) suitable for k-NN.

    b) Justify your choice of distance metric and any weighting scheme, taking into account the nature (continuous, ordinal, etc.) of each feature.

    c) Outline a validation framework (cross-validation strategy, performance metrics, hyper-parameter search) to select the optimal k and any other tunable parameters.

4 3. Algorithm Steps

4.1 Algorithmic Procedure for k-Nearest Neighbours Regression

In the preceding discussion we emphasized that k-NN regression is a *non-parametric, lazy* learning method: the training data are retained verbatim, and all model-building work

is postponed until a query point $\mathbf{x}_q$ is presented. The practical implementation therefore consists of three explicit steps that must be carried out for every prediction. We now describe these steps in detail, introduce the necessary notation, and discuss the rationale behind each operation.

4.1.1 Notation

- Training set: $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N}$, where $\mathbf{x}_i \in \mathbb{R}^p$ denotes a p-dimensional feature vector and $y_i \in \mathbb{R}$ the associated continuous target.
- Query (test) point: $\mathbf{x}_q \in \mathbb{R}^p$.
- Distance function: $d(\mathbf{x}_a, \mathbf{x}_b)$ measures similarity between two feature vectors. The most common choice is the Euclidean distance

$$d_{\text{Euc}}(\mathbf{x}_a, \mathbf{x}_b) = \left\|\mathbf{x}_a - \mathbf{x}_b\right\|_2 = \sqrt{\sum_{j=1}^{p} \left(x_{a,j} - x_{b,j}\right)^2}.$$

- Ordered distances: after computing distances from $\mathbf{x}_q$ to every training point, we sort them in ascending order and denote the indices of the k smallest distances by $(i_{(1)}, i_{(2)}, \ldots, i_{(k)})$. Consequently,

$$d\Big(\mathbf{x}_q, \mathbf{x}_{i_{(1)}}\Big) \leq d\Big(\mathbf{x}_q, \mathbf{x}_{i_{(2)}}\Big) \leq \cdots \leq d\Big(\mathbf{x}_q, \mathbf{x}_{i_{(k)}}\Big).$$

- Corresponding target values: $y_{(j)} = y_{i_{(j)}}$ for $j = 1, \ldots, k$.

4.1.2 Step 1 – Distance Computation

For a given query point $\mathbf{x}_q$ we evaluate the distance to each training observation:

$$\forall i \in \{1, \ldots, N\}: \quad d_i = d(\mathbf{x}_q, \mathbf{x}_i).$$

Because k-NN is *instance-based*, this computation is the only operation that involves the training set at prediction time. The choice of distance metric reflects the underlying assumption of **local smoothness**: points that are close in the feature space are assumed to have similar target values.

4.1.3 Step 2 – Selection of the k Nearest Neighbors

After obtaining the full distance vector $(d_1, \ldots, d_N)$, we identify the k smallest entries. Formally,

$$\mathcal{N}_k(\mathbf{x}_q) = \{i_{(1)}, i_{(2)}, \ldots, i_{(k)}\},$$

where $\mathcal{N}_k(\mathbf{x}_q)$ denotes the *neighbourhood* of $\mathbf{x}_q$. Several practical considerations arise:

1. **Tie handling**: if multiple training points share the same distance at the boundary, a deterministic tie-breaking rule (e.g., random selection or ordering by index) must be specified.
2. **Choice of k**: small k yields low bias but high variance, whereas large k smooths the estimate at the cost of increased bias. The optimal k is typically selected by cross-validation.

3. **Weighted neighbourhoods**: a variant replaces the simple count-based neighbourhood by a weight function $w_i = K\big(d_i/h\big)$ (e.g., Gaussian kernel). In the basic formulation presented here the weights are uniform, i.e., $w_i = 1/k$ for all neighbours.

4.1.4 Step 3 – Prediction by Averaging

Having isolated the k nearest training targets $\{y_{(1)}, \ldots, y_{(k)}\}$, the regression estimate for the query point is the arithmetic mean:

$$\boxed{\hat{y}_q \;=\; \frac{1}{k}\sum_{j=1}^{k} y_{(j)}}$$

This estimator is the maximum-likelihood estimate of the conditional mean $\mathbb{E}[Y \mid \mathbf{X} = \mathbf{x}_q]$ under the (implicit) assumption that the target values within the neighbourhood are i.i.d. draws from a distribution with constant mean.

4.1.5 Algorithmic Summary

Putting the three steps together, the complete prediction routine for a single query point can be expressed algorithmically:

Input: $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N},\ \mathbf{x}_q,\ k,$ distance $d(\cdot,\cdot)$.

1. Compute distances: $d_i \leftarrow d(\mathbf{x}_q, \mathbf{x}_i), \quad i = 1, \ldots, N.$

2. Identify neighbours: Find indices $(i_{(1)}, \ldots, i_{(k)})$ with smallest d_i.

3. Average targets: $\hat{y}_q \leftarrow \dfrac{1}{k}\displaystyle\sum_{j=1}^{k} y_{i_{(j)}}.$

Output: $\hat{y}_q$.

4.1.6 Connection to the Non-Parametric Paradigm

Because the algorithm never collapses the training data into a fixed set of parameters, its expressive power grows with the amount of data available. The model's complexity is therefore *data-driven*: with more observations, finer local structure can be captured without altering the algorithmic form. This property distinguishes k-NN regression from parametric alternatives (e.g., linear regression), and aligns with the overarching theme that non-parametric methods adapt their flexibility to the training set.

4.1.7 Practical Remarks

- **Computational cost**: The naïve implementation requires $O(Np)$ operations per query (distance to every training point). For large datasets, acceleration structures such as kd-trees or ball trees can reduce the average cost, especially in low-to-moderate dimensions.

- **Feature scaling**: Since Euclidean distance is sensitive to the relative scale of features, preprocessing steps such as standardization (zero mean, unit variance) or min-max scaling are essential to avoid domination of some dimensions.

- **Curse of dimensionality**: As p grows, distances tend to concentrate, weakening the discriminatory power of the nearest-neighbour rule. Dimensionality reduction (PCA, manifold learning) or distance-metric learning may be employed to mitigate this effect.

The three-step procedure outlined above constitutes the operational core of k-NN regression and will serve as the basis for subsequent sections on model selection, complexity analysis, and extensions to weighted and adaptive neighbourhood schemes.

Questions

1. **Numerical prediction (PYQ).** Given the training data $\{(2, 4), (5, 8), (8, 12), (11, 16)\}$ and a query point $x = 7$ with $k = 3$:
   (a) Compute the Euclidean distance between the query point and each training sample.
   (b) Select the three training samples with the smallest distances.
   (c) Predict the target $\hat{y}$ as the average of the y-values of the selected neighbours.
2. **Algorithmic description and metric change.**
   (a) Write down the three-step algorithm for k-NN regression, explicitly indicating how the distance is computed and how the final prediction $\hat{y}$ is obtained.
   (b) Discuss how replacing the Euclidean distance by the Manhattan distance would affect neighbour selection for a query point in a two-dimensional feature space. Illustrate with a brief sketch or numerical example.
3. **Effect of feature scaling.**
   (a) Explain qualitatively how Euclidean distance is distorted when the two features age $\in [0, 100]$ and income $\in [10\,000, 200\,000]$ are left in their original units.
   (b) Propose a preprocessing pipeline that ensures the two features contribute equally to distance calculations.
   (c) Construct a small numerical example with at least three training points showing how the predicted $\hat{y}$ for a given query changes before and after scaling.
4. **Tie-handling strategies.**
   (a) Compare the consequences of three deterministic tie-breaking rules when several points share the same distance at the k-th boundary: (i) choose the smallest index, (ii) pick at random, (iii) average the target values of all tied points.
   (b) For a production system where reproducibility is critical, which rule would you recommend and why?
5. **Naïve computational cost analysis.**
   (a) Derive the exact number of elementary operations (additions, multiplications, square-roots) required to compute all Euclidean distances for a single query when the training set has size N and dimensionality p.

(b) Suppose the training set resides on secondary storage and only a block of size B can be loaded into RAM at a time. Propose an I/O-efficient algorithm for distance computation.

(c) Estimate the total I/O cost (in block reads) for a dataset with $N = 10^8$ points and block size $B = 10^4$.

6. **Algorithm design with kd-tree acceleration.**

(a) Write pseudo-code that implements the three-step k-NN regression algorithm using a kd-tree to speed up neighbour search.

(b) Analyse the expected time complexity of a single query under the assumption that the data are uniformly distributed in a bounded region of $\mathbb{R}^p$.

(c) Explain why the advantage of a kd-tree diminishes as the dimensionality p grows, linking your discussion to the curse of dimensionality.

7. **Bias–variance trade-off for different k.**

(a) Sketch (or describe) how the expected squared error $\mathbb{E}\big[(\hat{Y} - Y)^2\big]$ decomposes into bias^2 and variance components as a function of the neighbourhood size k for the one-dimensional regression model $f(x) = \sin(2\pi x)$ with i.i.d. Gaussian noise.

(b) Explain why very small k yields low bias but high variance, whereas large k produces the opposite effect.

(c) Propose a data-driven method for selecting an optimal k and outline the steps required to implement it (e.g., K-fold cross-validation).

8. **Weighted neighbourhoods with a Gaussian kernel.**

(a) Derive the limiting behaviour of the weighted estimator

$$\hat{y}_q = \frac{\sum_{i \in \mathcal{N}_k(\mathbf{x}_q)} w_i \, y_i}{\sum_{i \in \mathcal{N}_k(\mathbf{x}_q)} w_i}, \qquad w_i = \exp\!\Big(-\tfrac{d_i^2}{2h^2}\Big),$$

as the bandwidth $h \to 0$ and as $h \to \infty$.

(b) Discuss how the choice of h interacts with the choice of k. In particular, comment on scenarios where a large k combined with a small h might be advantageous (or not).

(c) Design a cross-validation scheme to jointly tune the pair (k, h). Specify the search strategy and evaluation metric.

9. **Empirical investigation of distance concentration.**

(a) Generate synthetic data uniformly in the hyper-cube $[0, 1]^p$ for several values of p (e.g., $p = 2, 5, 10, 20, 50$).

(b) For each dimensionality, compute the ratio

$$R(p) = \frac{\text{mean distance to nearest neighbour}}{\text{mean distance to farthest neighbour}}.$$

(c) Describe (or sketch) how $R(p)$ evolves with p and interpret the result with respect to the reliability of the nearest-neighbour rule in high dimensions.

10. **Adaptive neighbourhood size (radius-based).**

    (a) Discuss the advantages and disadvantages of defining the neighbourhood as all points within a radius r of the query rather than fixing k.

    (b) Propose an automatic rule for selecting r for each query point (e.g., based on a quantile of the distance distribution or on a density estimate).

    (c) Rewrite the algorithmic summary to accommodate a radius-based neighbourhood, indicating where the additional computation appears.

11. **Non-parametric vs. parametric modelling.**

    (a) Explain, in the context of the three-step algorithm, why k-NN is classified as a non-parametric method.

    (b) Contrast the learning bias of k-NN regression with that of ordinary least-squares linear regression when applied to the same dataset.

    (c) Identify circumstances (e.g., data size, noise level, underlying functional form) under which a parametric model might outperform k-NN despite the latter's greater flexibility.

5 4. Advantages

5.1 Desirable Properties of the k-Nearest Neighbours Regressor

In the preceding discussion we introduced the k-Nearest Neighbours (k-NN) regressor as a lazy, non-parametric learner whose prediction for a query point $\mathbf{x}_q$ is obtained by

$$\hat{y}(\mathbf{x}_q) = \frac{1}{k} \sum_{i \in \mathcal{N}_k(\mathbf{x}_q)} y_i,$$

where $\mathcal{N}_k(\mathbf{x}_q)$ denotes the set of the k training instances that are closest to $\mathbf{x}_q$ according to a chosen distance function. Beyond its conceptual simplicity, k-NN enjoys a number of practical advantages that make it a useful baseline and, in many settings, a competitive modelling choice. These advantages are summarised in the four pillars below.

5.2 Simplicity

- **No model training.** The algorithm does not require the estimation of parameters by optimisation; the entire training set $\{(\mathbf{x}_i, y_i)\}_{i=1}^{n}$ is stored unchanged. Consequently, the *training phase* reduces to a memory allocation step, and all computational effort is concentrated in the *prediction phase*.

- **Pure distance calculations.** For a query $\mathbf{x}_q$ we compute a vector of distances

  $$d_i = \mathrm{dist}(\mathbf{x}_q, \mathbf{x}_i), \qquad i = 1, \ldots, n,$$

  rank them, and select the smallest k. No gradient descent, regularisation terms, or matrix inversions are required.

5.3 Versatility

- **Continuous and categorical targets.** The same neighbour-selection machinery can be used for regression (continuous y) and for classification (categorical y). In the latter case the prediction rule becomes a majority vote or a probability estimate derived from neighbour frequencies.

- **Mixed-type feature spaces.** Because the algorithm only relies on a distance measure, one can combine numerical attributes (using Euclidean or Mahalanobis distance) with categorical attributes (using Hamming distance or a specialised mixed-type metric) without altering the overall framework.

5.4 Interpretability

- **Direct link to observed samples.** The prediction $\hat{y}(\mathbf{x}_q)$ is explicitly expressed as an average (or weighted average) of the actual response values $\{y_i\}$ of the selected neighbours. Consequently, a practitioner can inspect the contributing training points to understand why a particular estimate was produced.

- **Transparent influence of neighbours.** If a weighting scheme is employed (see Flexibility below), the contribution of each neighbour is given by a weight w_i that is usually a monotone function of the distance d_i. The final estimate can then be written as

$$\hat{y}(\mathbf{x}_q) = \frac{\sum_{i \in \mathcal{N}_k(\mathbf{x}_q)} w_i \, y_i}{\sum_{i \in \mathcal{N}_k(\mathbf{x}_q)} w_i},$$

  making it straightforward to trace the effect of each neighbour on the outcome.

5.5 Flexibility

- **Choice of distance metric.** The definition of $\mathrm{dist}(\cdot, \cdot)$ is not fixed. Common options include

$$\begin{aligned} \text{Euclidean:} \quad & d_i = \|\mathbf{x}_q - \mathbf{x}_i\|_2, \\ \text{Manhattan:} \quad & d_i = \|\mathbf{x}_q - \mathbf{x}_i\|_1, \\ \text{Mahalanobis:} \quad & d_i = \sqrt{(\mathbf{x}_q - \mathbf{x}_i)^\top \boldsymbol{\Sigma}^{-1} (\mathbf{x}_q - \mathbf{x}_i)}, \end{aligned}$$

  where $\boldsymbol{\Sigma}$ denotes the covariance matrix of the features. Custom distances can be designed to respect domain-specific similarity notions.

- **Weighting schemes.** Rather than a simple arithmetic mean, one may assign larger weights to nearer neighbours. A popular choice is the *inverse-distance* weight

$$w_i = \frac{1}{d_i^\alpha + \epsilon},$$

  with exponent $\alpha > 0$ controlling the decay rate and ϵ preventing division by zero. Kernel-based weights (e.g., Gaussian kernel $w_i = \exp(-d_i^2 / 2h^2)$) provide smooth attenuation.

- **Adaptive neighbourhood size.** The parameter k can be fixed globally, selected by cross-validation, or even varied locally (e.g., choosing the smallest radius that contains a preset total weight). This adaptability permits the model to handle regions of differing data density.

5.6 Summary

The k-NN regressor epitomises a learning paradigm in which the complexity of the model grows with the amount of data, while the algorithmic core remains elementary: compute distances, pick the nearest neighbours, and aggregate their observed responses. Its simplicity, versatility across target types, transparent reliance on actual observations, and the freedom to tailor distance and weighting schemes collectively render it a powerful baseline and a pedagogical tool for understanding more sophisticated non-parametric methods.

Questions

1. **System-level latency (Simplicity)**

   (a) Explain how the "no model training" property of k-NN influences overall system latency compared with a linear-regression model, addressing both the training and prediction phases.

   (b) Give a quantitative illustration showing when k-NN can become slower at prediction time despite its simplicity. Assume n training points and k neighbours; express the prediction-time complexity and discuss realistic values of n and k where the cost is prohibitive.

2. **Interpretability claim (Simplicity)** Critically assess the statement: "Because k-NN requires only distance calculations, it is always more interpretable than a parametric model such as ridge regression." Discuss at least two scenarios in which this claim may fail.

3. **Mixed-type distance and classification (Versatility)**

   (a) Propose a mixed-type distance metric for a dataset containing numeric attributes (e.g., age, income) and a categorical attribute (region $\in \{\text{North}, \text{South}, \text{East}, \text{West}\}$). Write the full mathematical form and justify each component.

   (b) If the target variable were categorical (e.g., "buy" vs. "not buy"), describe how the neighbour-selection/prediction step would be modified.

4. **Averaging probabilities (Versatility)** When the response variable represents a probability (values in $[0, 1]$), explain why the basic un-weighted average of neighbour responses is appropriate. Identify a situation where a different aggregation (e.g., weighted majority) would be preferable and justify your choice.

5. **Weighted prediction calculation (Interpretability)** For a query point $\mathbf{x}_q$ the four nearest neighbours have

i	1	2	3	4
y_i	3	7	5	9
d_i	0.2	0.5	0.3	0.6

(a) Compute the prediction using (a) simple arithmetic mean and (b) inverse-distance weighting with $\alpha = 1$ and $\epsilon = 10^{-5}$.

(b) Discuss how the weighting scheme alters the interpretability of the prediction relative to the simple mean. Which neighbour exerts the greatest influence and why?

6. **Generating a "why-this-prediction" explanation (Interpretability)** Outline a step-by-step procedure a data scientist could follow to produce a transparent explanation for a k-NN credit-risk score. Highlight any residual limitations of the explanation.

7. **Custom distance with clinical importance (Flexibility)** Design a distance metric for a medical dataset where each feature j has an associated clinical importance weight $w_j > 0$. Provide the formula and describe a practical way to calibrate the weights using domain expertise.

8. **Comparing weighting schemes (Flexibility)** Contrast the following two weighting functions:

$$w_i^{\text{inv}} = \frac{1}{d_i^{\alpha} + \epsilon}, \qquad w_i^{\text{gauss}} = \exp\!\Big(-d_i^2/(2h^2)\Big).$$

Discuss their respective impacts on the bias–variance trade-off and indicate which is more robust to outliers in the distance distribution.

9. **Adaptive neighbourhood selection (Flexibility)** Propose an adaptive strategy that chooses the local neighbourhood size $k(\mathbf{x}_q)$ so that the total accumulated weight reaches a target W_{target}. Provide concise pseudo-code and explain how the method mitigates problems caused by heterogeneous sample density.

10. **Effect of increasing k (Flexibility)** A researcher claims that larger values of k always reduce over-fitting. Construct an argument, supported by a simple illustrative example, that demonstrates a case where increasing k actually degrades predictive performance.

11. **Basic k-NN regression calculation (Past-Year Question)** Using the training data below, predict the target value for a new observation with $x = 7$ and $k = 3$. Euclidean distance is used and the prediction is the average of the three nearest neighbours' y-values.

x	y
2	4
5	8
8	12
11	16

Show all steps (distance computation, neighbour selection, averaging) and state the final predicted value.

6 5. Limitations

6.1 Disadvantages of k-Nearest Neighbours Regression

Although k-NN regression is attractive for its simplicity and its non-parametric, lazy-learning nature, several practical and theoretical limitations arise from the way the algorithm operates. In this section we elaborate on the five most important drawbacks that follow directly from the raw observations listed above.

6.1.1 Bias–Variance Trade-off Governed by k

For a given query point $\mathbf{x}$ the k-NN predictor is

$$\hat{y}(\mathbf{x}) \;=\; \frac{1}{k} \sum_{i \in \mathcal{N}_k(\mathbf{x})} y_i,$$

where $\mathcal{N}_k(\mathbf{x})$ denotes the index set of the k nearest training samples with responses y_i. The expected squared error can be decomposed as

$$\mathbb{E}\Big[(\hat{y}(\mathbf{x}) - f(\mathbf{x}))^2\Big] = \underbrace{\Big(\mathbb{E}[\hat{y}(\mathbf{x})] - f(\mathbf{x})\Big)^2}_{\text{Bias}^2} + \underbrace{\mathbb{E}\Big[(\hat{y}(\mathbf{x}) - \mathbb{E}[\hat{y}(\mathbf{x})])^2\Big]}_{\text{Variance}} + \sigma_\varepsilon^2,$$

where $f(\mathbf{x})$ is the true regression function and σ_ε^2 the irreducible noise. Choosing a small k makes the neighbourhood narrow, so the estimator follows the data closely (low bias) but the average is taken over few noisy observations (high variance). Conversely, a large k smooths over many points, reducing variance at the cost of increasing bias. Hence the user must balance these two opposing forces, and there is no universal rule for the optimal k.

6.1.2 Sensitivity to Outliers and Noise

Because the prediction is a simple arithmetic mean of the raw target values of the nearest neighbours, any anomalous observation that lies among the k closest points can exert a disproportionate influence. If an outlier has response y_{out} that deviates markedly from the surrounding values, the resulting prediction becomes

$$\hat{y}(\mathbf{x}) = \frac{1}{k}\Big(y_{\text{out}} + \sum_{j \neq \text{out}} y_j\Big),$$

so even a single outlier can shift $\hat{y}(\mathbf{x})$ by $\frac{1}{k}(y_{\text{out}} - \bar{y}_{\text{rest}})$. The effect diminishes only when k is large, which, as discussed above, increases bias. Moreover, the distance-based neighbourhood selection itself is vulnerable: noisy features can alter Euclidean (or other) distances, causing the algorithm to select inappropriate neighbours.

6.1.3 Computational Burden at Prediction Time

k-NN is a *lazy* learner: all training instances $\{(\mathbf{x}_i, y_i)\}_{i=1}^N$ are retained, and no model is fitted ahead of time. To predict at a single query point the algorithm must compute the distance

$$d(\mathbf{x}, \mathbf{x}_i) = \|\mathbf{x} - \mathbf{x}_i\|_2 \qquad i = 1, \ldots, N,$$

which entails $\mathcal{O}(N)$ operations. For Q queries the total cost is $\mathcal{O}(QN)$. When N is large (e.g., millions of training records) the naïve approach becomes prohibitive both in time and memory, motivating the use of approximate nearest-neighbour structures (KD-trees, ball-trees, locality-sensitive hashing) that trade exactness for speed.

6.1.4 Curse of Dimensionality

In high-dimensional spaces the notion of "neighbourhood" degrades because distances concentrate. Let $\mathbf{x}$ and $\mathbf{x}'$ be drawn i.i.d. from a distribution with bounded support. For a Euclidean metric, the ratio between the expected maximum and minimum distances among the N points satisfies

$$\frac{\mathbb{E}[\max_i d(\mathbf{x}, \mathbf{x}_i)]}{\mathbb{E}[\min_i d(\mathbf{x}, \mathbf{x}_i)]} \xrightarrow[d\to\infty]{} 1,$$

where d denotes the dimensionality. Consequently, all points become almost equidistant, and the algorithm loses its ability to discriminate truly "near" neighbours from distant ones. The effective neighbourhood therefore expands, inflating bias and worsening predictive performance. Dimensionality reduction or feature selection is often required before applying k-NN in such settings.

6.1.5 Impact of Region Imbalance

Although we are focusing on regression, the same geometric intuition applies to classification and to the density of training points in different regions of the input space. If the data are imbalanced, i.e., some regions contain far more samples than others, the nearest-neighbour search will preferentially select points from the dense region. For a query $\mathbf{x}$ situated near the boundary between a dense and a sparse region, the majority of its k neighbours will belong to the dense side, leading the averaged prediction to be biased toward that region's local mean. Formally, let $p_{\text{dense}} \gg p_{\text{sparse}}$ denote the local sampling densities; the expected composition of $\mathcal{N}_k(\mathbf{x})$ is weighted by these densities, and the estimator becomes

$$\mathbb{E}[\hat{y}(\mathbf{x})] \approx \frac{p_{\text{dense}}\,\mu_{\text{dense}} + p_{\text{sparse}}\,\mu_{\text{sparse}}}{p_{\text{dense}} + p_{\text{sparse}}},$$

where μ_{dense} and μ_{sparse} are the local conditional means. When p_{dense} dominates, the prediction is pulled toward μ_{dense}, regardless of the true underlying function at $\mathbf{x}$.

6.1.6 Summary

The five limitations above—(i) the bias–variance dependence on k, (ii) vulnerability to outliers and noise, (iii) linear-in-N computational cost at query time, (iv) degradation of distance discrimination in high dimensions, and (v) bias introduced by uneven sampling densities—constitute the primary challenges that practitioners must address when deploying k-NN regression. Understanding these drawbacks informs the choice of complementary techniques (e.g., robust distance metrics, dimensionality reduction, approximate neighbour structures, or adaptive k) that can mitigate the weaknesses while preserving the algorithm's intuitive appeal.

Questions

1. **Basic computation.** Using the training set $\{(2,4),\ (5,8),\ (8,12),\ (11,16)\}$, predict the target value for a new sample with $x = 7$ when $k = 3$. Show all distance calculations and the final averaged prediction.

2. **Bias–variance trade-off and model selection (medium).** You are given a regression data set with 10000 training points and a separate validation set.

   (a) Propose a systematic procedure—including a concrete cross-validation scheme—to select an optimal k.

   (b) For the three candidate values $k = 1$, $k = 50$, $k = 500$ describe how you would use the bias–variance decomposition to interpret the observed validation errors.

   (c) Discuss the risks of adopting the value of k that yields the lowest validation error if it is far from the theoretical bias–variance optimum.

3. **Outlier influence (medium).** A one-dimensional query point x^* has the three nearest neighbours (ordered by distance) with responses $y_1 = 10,\ y_2 = 12,\ y_3 = 30$.

   (a) Compute the predicted value $\hat{y}(x^*)$ for $k = 1, 2, 3$.

   (b) Identify the outlier and quantify its influence on each prediction using $\frac{1}{k}\big(y_{\text{out}} - \bar{y}_{\text{rest}}\big)$.

   (c) Propose two robust modifications to the vanilla k-NN predictor that would reduce the impact of such an outlier, and discuss the associated trade-offs.

4. **Complexity analysis (hard).** You must answer 1 000 000 queries on a data set containing $N = 5 \times 10^6$ training points.

   (a) Estimate the total number of distance calculations required by the naïve implementation.

   (b) Give the asymptotic query-time complexity of the naïve method versus a balanced KD-tree implementation, and explain in which regimes (dimensionality, data distribution) the KD-tree advantage diminishes.

   (c) Design a hybrid search strategy that combines exact and approximate neighbour retrieval to meet a strict latency budget while keeping prediction error within a pre-specified tolerance.

5. **Curse of dimensionality (hard).** Let each feature be drawn i.i.d. from a uniform distribution on $[0,1]^d$.

   (a) Sketch a proof that $\dfrac{\mathbb{E}[\max_i d(\mathbf{x}, \mathbf{x}_i)]}{\mathbb{E}[\min_i d(\mathbf{x}, \mathbf{x}_i)]} \to 1$ as $d \to \infty$.

   (b) Propose a simulation experiment to illustrate how the mean absolute prediction error of a 5-NN regressor evolves as d varies from 2 to 100, keeping the number of training points fixed. State which metrics you would record and the expected trend.

   (c) Discuss two preprocessing techniques that can alleviate the curse of dimensionality for k-NN regression, and explain how each technique changes the distance-concentration behavior.

6. **Impact of region imbalance (medium–hard).** Training data are sampled from two regions: a dense region with density $p_{\text{dense}} = 0.9$ and local mean $\mu_{\text{dense}} = 50$, and a sparse region with density $p_{\text{sparse}} = 0.1$ and local mean $\mu_{\text{sparse}} = 20$.

   (a) Derive the expected prediction $\mathbb{E}[\hat{y}(\mathbf{x})]$ for a query point that lies exactly on the boundary of the two regions when $k = 10$.

   (b) Explain how increasing k biases the prediction toward the dense region; quantify this effect for $k = 10$ versus $k = 100$.

   (c) Propose a weighting scheme (e.g., distance-based, density-adjusted) that reduces the bias caused by the imbalance, and outline a validation protocol to assess its efficacy on a held-out test set.

7. **Integrated real-time deployment scenario (hard).** You are asked to deploy a real-time k-NN regression model for estimating vehicle fuel consumption from 30 sensor readings. The historical archive contains 2million trips, occasional sensor glitches introduce outliers, and many sensors are highly correlated.

   (a) Design a step-by-step pipeline that tackles each of the five disadvantages discussed (bias–variance, outliers, computational cost, dimensionality, imbalance). For each step indicate the chosen technique (e.g., adaptive k selection, robust distance metric, KD-tree / LSH, PCA, density-aware weighting).

   (b) For every pipeline component specify a measurable performance indicator (e.g., validation RMSE, 99-th-percentile latency, outlier-influence score) and a target threshold that would justify the technique's adoption.

   (c) Discuss a post-deployment monitoring and adaptation plan to handle data-distribution drift (new vehicle models, sensor upgrades), including what statistics you would track and how you would trigger retraining or hyper-parameter re-tuning.

7 Worked Example

7.1 Theory

Consider the one-dimensional regression problem with training set

$$\mathcal{D} = \{(x_1, y_1) = (2, 4),\ (x_2, y_2) = (5, 8),\ (x_3, y_3) = (8, 12),\ (x_4, y_4) = (11, 16)\}.$$

For a query point $x_q = 7$ and a k-Nearest Neighbours (k-NN) regressor with $k = 3$ we proceed through three steps:

1. **Distance computation.** In $\mathbb{R}^1$ the natural distance is the absolute difference

$$d(x_q, x_i) = |x_q - x_i|, \qquad i = 1, \ldots, 4.$$

   Evaluating:

i	x_i	$d(x_q, x_i)$
1	2	5
2	5	2
3	8	1
4	11	4

2. **Neighbour selection.** Ordering the distances gives the three nearest neighbours
$$\mathcal{N}_3(x_q) = \{(5, 8),\ (8, 12),\ (11, 16)\}.$$
3. **Prediction by averaging.** The k-NN regression estimator is the mean of the neighbour targets:
$$\hat{y}(x_q) = \frac{1}{k} \sum_{i \in \mathcal{N}_k(x_q)} y_i = \frac{1}{3}(8 + 12 + 16) = 12.$$

The procedure illustrates the three core operations of k-NN regression (distance calculation, neighbour retrieval, and target averaging) and provides a spring-board for analysing bias/variance, robustness, and comparisons with parametric models.

7.2 Questions

1. **Basic prediction (replicating the worked example).**
Using the training data $\mathcal{D}$ above, compute the k-NN regression estimate for the query point $x_q = 7$ with $k = 3$. Show the distance calculations, neighbour selection and the final average.

2. **Prediction at a different query point.**
Compute the k-NN regression estimate for $x_q = 6$ with $k = 3$. Provide all intermediate steps (distances, neighbour set, averaging) and compare the result with the estimate $\hat{y}(7) = 12$ obtained earlier. Comment on why the predictions differ.

3. **Effect of an alternative distance measure.**
Suppose the distance is scaled by a constant factor $w = 0.5$:
$$d_w(x_q, x_i) = w\,|x_q - x_i|.$$
   (a) Re-evaluate the distances for $x_q = 7$ and identify the three nearest neighbours under the scaled metric.
   (b) Compute the corresponding prediction $\hat{y}_w(7)$ and compare it with the original $\hat{y}(7) = 12$.
   (c) Explain why multiplying the distance by a positive constant does (or does not) change the neighbour set and the resulting prediction.

4. **Bias–variance trade-off for different k.**
Using the original data set, qualitatively describe the expected bias and variance of the k-NN regressor for:
   (a) a small value of k (e.g. $k = 1$);
   (b) a large value of k (e.g. $k = 4$).

   Relate your discussion to the notions of under-fitting and over-fitting.

5. **Assumptions underlying simple averaging.**
List the statistical assumptions that make the plain average of the neighbour targets an unbiased estimator of $\mathbb{E}[Y \mid X = x_q]$. For each assumption, discuss what could happen if it is violated in practice (e.g., heteroscedastic noise, dependence between observations).

6. **Influence of an outlier.**
   Assume the fourth training point is corrupted to $(x_4, y_4) = (11, 30)$.
   (a) With $k = 3$ recompute the prediction at $x_q = 7$.
   (b) Analyse how this outlier changes the estimate compared with the outlier-free case.
   (c) Propose a simple modification to the k-NN algorithm that could mitigate the impact of such outliers (e.g., robust aggregation, distance weighting, or outlier detection).

7. **Non-zero-mean noise scenario.**
   Suppose the observed targets follow $Y = f(X) + \varepsilon$ where ε has mean $\mu_\varepsilon \neq 0$.
   (a) Explain how a non-zero mean noise term biases the k-NN regression estimate.
   (b) Suggest a practical strategy to obtain an unbiased estimate of the underlying function $f(x)$ when μ_ε is unknown.

8. **Comparison with a parametric alternative.**
   Fit a simple linear regression model $y = ax + b$ to the training data $\mathcal{D}$ by ordinary least squares.
   (a) Determine the optimal coefficients a and b.
   (b) Compute the linear-model prediction at $x_q = 7$ and compare it with the k-NN prediction $\hat{y}(7) = 12$.
   (c) Discuss situations in which the k-NN regressor would be preferred over this linear model, and vice-versa, for the given data set.

9. **Choosing the optimal k.**
   Outline a data-driven procedure (e.g., K-fold cross-validation) for selecting the best value of k based on the available training set. Include:
   (a) The performance metric to optimise (e.g., mean-squared error).
   (b) How the data would be split into training/validation folds and why.
   (c) Potential pitfalls of using the same data both for neighbour selection and for performance evaluation, and how to avoid them.

10. **Interpretation of the prediction.**
    (a) Express $\hat{y}(7) = 12$ as an empirical estimate of a conditional expectation.
    (b) Discuss what this estimate tells us about the underlying relationship between X and Y in the neighbourhood of $x \approx 7$.
    (c) Explain how the interpretation would change if the neighbourhood were weighted (e.g., using inverse-distance weighting) rather than a simple unweighted average.

8 7. Summary

8.1 Parametric vs. Non-Parametric Learning

A learning algorithm can be classified according to how the hypothesis space grows with the amount of training data.

8.1.1 Parametric models

A *parametric* model assumes a fixed-size set of parameters $\boldsymbol{\theta} \in \mathbb{R}^p$ that completely determines the predictor

$$\hat{y} = f(\mathbf{x}; \boldsymbol{\theta}).$$

The number of parameters p does not depend on the number of training examples N. Classical examples are linear regression ($p = d + 1$), logistic regression, and neural networks with a prescribed architecture. Because the model capacity is limited, parametric learners tend to exhibit a *bias* that is independent of N but a variance that typically *decreases* as more data are observed.

8.1.2 Non-parametric models

A *non-parametric* model does not assume a predefined parametric form. Instead, the complexity of the hypothesis grows with the training set. Formally,

$$\hat{y} = g_N(\mathbf{x}; \mathcal{D}_N),$$

where $\mathcal{D}_N = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$ is the entire training collection and the effective number of degrees of freedom is a function of N. The bias of a non-parametric learner can vanish as $N \to \infty$; however, its variance may increase because the model *stores* information about each training point.

8.1.3 Bias–Variance intuition

$$\mathrm{E}\left[(\hat{y} - y)^2\right] = \underbrace{(\mathrm{Bias}[\hat{y}])^2}_{\text{systematic error}} + \underbrace{\mathrm{Var}[\hat{y}]}_{\text{estimation error}} + \sigma_\varepsilon^2,$$

where σ_ε^2 is the irreducible noise. Parametric models usually have larger bias, smaller variance; non-parametric models have the opposite tendency. Selecting an appropriate family therefore depends on the available data, the desired interpretability, and the computational budget.

8.2 Key Practical Considerations When Building Machine-Learning Models

1. **Feature relevance and scaling**
   Non-parametric distance-based methods (e.g., k-NN) are extremely sensitive to irrelevant or differently scaled features. Typical remedies:

   - Standardisation: $x'_j = (x_j - \mu_j)/\sigma_j$.
   - Normalisation to unit range: $x'_j = (x_j - \min_j)/(\max_j - \min_j)$.

- Feature selection or dimensionality reduction (e.g., PCA) to mitigate the curse of dimensionality.

2. **Choice of distance metric**
   Euclidean distance
   $$d_{\mathrm{E}}(\mathbf{x}, \mathbf{x}') = \left\|\mathbf{x} - \mathbf{x}'\right\|_2$$
   is standard, but alternative metrics (Manhattan, Mahalanobis) may better capture the geometry of the data, especially when features are correlated.

3. **Model capacity vs. computational cost**

   - *Training time*: Parametric learners usually require an explicit optimisation (e.g., gradient descent). Non-parametric learners such as k-NN have essentially no training cost.
   - *Inference time*: For k-NN the naïve prediction cost is $O(Nd)$ per query (computing distances to all N points in d dimensions). Indexing structures (KD-trees, ball-trees) can reduce this cost for low-to-moderate dimensions.

4. **Robustness to noise**
   Averaging over many neighbours reduces variance but may also smooth away sharp local structure. Picking an appropriate k is therefore a bias–variance trade-off.

5. **Cross-validation and hyper-parameter selection**
   The neighbourhood size k and any distance-metric parameters must be chosen by a validation procedure (e.g., K-fold CV) to avoid over-fitting the validation set.

6. **Curse of dimensionality**
   In high dimension the volume of a sphere grows so fast that the nearest neighbours become almost as far away as any random point. Consequently
   $$\frac{d(\mathbf{x}, \text{nearest})}{d(\mathbf{x}, \text{farthest})} \to 1 \quad \text{as } d \to \infty,$$
   which destroys the notion of "locality". Dimensionality reduction or parametric alternatives are often preferable when d is large.

8.3 k-Nearest Neighbour Regression

8.3.1 Definition

Given a query point $\mathbf{x}_q \in \mathbb{R}^d$ and a training set $\mathcal{D}_N = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$, the *k-NN regressor* predicts

$$\boxed{\hat{y}(\mathbf{x}_q) = \frac{1}{k} \sum_{i \in \mathcal{N}_k(\mathbf{x}_q)} y_i} \tag{1}$$

where $\mathcal{N}_k(\mathbf{x}_q)$ denotes the indices of the k training points with smallest distance $d(\mathbf{x}_q, \mathbf{x}_i)$.

8.3.2 Weighted variants

A common refinement is to give nearer neighbours larger influence:

$$\hat{y}(\mathbf{x}_q) = \frac{\sum_{i \in \mathcal{N}_k(\mathbf{x}_q)} w_i \, y_i}{\sum_{i \in \mathcal{N}_k(\mathbf{x}_q)} w_i}, \qquad w_i = K\Big(d(\mathbf{x}_q, \mathbf{x}_i)\Big),$$

where $K(\cdot)$ is a kernel function (e.g., Gaussian $K(r) = \exp(-r^2/2h^2)$). The bandwidth h controls the degree of smoothing.

8.3.3 Algorithmic steps

1. **Distance computation**: calculate $d(\mathbf{x}_q, \mathbf{x}_i)$ for all $i = 1, \ldots, N$.
2. **Neighbour selection**: sort the distances (or maintain a min-heap) and keep the indices of the k smallest values.
3. **Aggregation**: average the corresponding targets using (1) or a weighted scheme.

8.3.4 Worked example (univariate)

x_i	y_i
1.0	2.0
2.0	2.5
3.0	3.5
4.0	5.0
5.0	4.5

Predict at $x_q = 3.4$ with $k = 3$.

1. Distances: 2.4, 1.4, 0.4, 0.6, 1.6.
2. Nearest indices: $3, 4, 2$.
3. Unweighted prediction: $\hat{y} = \dfrac{3.5 + 5.0 + 2.5}{3} = 3.67$.
4. With Gaussian weighting ($h = 0.5$) the weighted prediction becomes ≈ 4.03, illustrating the effect of giving larger weight to nearer points.

8.4 Questions

Q0 Conceptual comparison (easy).
Explain why a parametric model typically exhibits higher *bias* but lower *variance* than a non-parametric model. Illustrate your answer with a concrete comparison between linear regression (parametric) and k-NN regression (non-parametric), referencing the bias–variance decomposition given above.

Q0 Bias–variance trade-off for k-NN (medium).
Consider the univariate true relationship $y = \sin(\pi x) + \varepsilon$, $\varepsilon \sim \mathcal{N}(0, 0.1^2)$, with training points drawn uniformly from $[0, 1]$.

(Sketch qualitatively how the expected bias and variance of a k-NN regressor evolve as k increases from 1 to N.

(If the primary objective is to obtain *smooth* predictions rather than the smallest pointwise error, what region of k would you select and why?

Q0 Feature scaling impact (easy).
A 3-dimensional dataset has feature ranges $[0, 1]$, $[0, 100]$ and $[0, 0.01]$.

(Show quantitatively why Euclidean distance is dominated by the second feature when no preprocessing is applied.

(Propose a preprocessing pipeline (including scaling/normalisation steps) that mitigates this dominance, and justify each step.

Q0 Choosing a distance metric (medium).
Two continuous attributes x_1 and x_2 are highly correlated.

(Discuss why plain Euclidean distance may be sub-optimal for k-NN regression in this situation.

(Derive the Mahalanobis distance for the two-dimensional case using the covariance matrix Σ and explain how it accounts for the correlation.

Q0 Computational analysis (hard).
Assume a training set of size $N = 10^5$ and dimensionality $d = 20$.

(Estimate the number of elementary arithmetic operations required to answer a single query using the naïve k-NN algorithm (ignore constant-factor overhead).

(If a KD-tree is employed, give the asymptotic query complexity and discuss for which values of d the tree actually provides a speed-up over the naïve approach.

Q0 Cross-validation design (medium).
Design a K-fold cross-validation procedure to tune jointly the neighbourhood size k and the Gaussian bandwidth h in a weighted k-NN regressor. Specify:

(A sensible grid of candidate (k, h) pairs.

(How to aggregate the validation errors across folds in order to select the optimal pair (k, h).

(Practical tricks to keep the computational cost reasonable.

Q0 Curse of dimensionality illustration (hard).
Derive the limit

$$\frac{d(\mathbf{x}, \text{nearest})}{d(\mathbf{x}, \text{farthest})} \longrightarrow 1 \quad \text{as } d \to \infty$$

for points drawn i.i.d. from a standard d-dimensional Gaussian distribution. Explain why this erodes the notion of "nearest neighbour" and list at least two remedies mentioned in the theory.

Q0 Weighted k-NN calculation (easy).
Using the training data from the worked example (Section 8.3), recompute the prediction at $x_q = 3.4$ with a Gaussian kernel of bandwidth $h = 0.3$.

(Show the three weights w_i for the selected neighbours.

(Compute the weighted prediction $\hat{y}_{\mathrm{w}}(3.4)$ and compare it with the unweighted result 3.67.

Q0 Simple k-NN prediction (easy).
Given the training set $\{(2, 4),\ (5, 8),\ (8, 12),\ (11, 16)\}$, predict the target for a new point $x = 7$ with $k = 3$ using ordinary (unweighted) k-NN regression. Show the distance calculations, the chosen neighbours, and the final prediction.

Q0 Model selection scenario (medium).
For a medical dataset with $N = 500$ patients and $d = 50$ laboratory measurements (many redundant), discuss whether a parametric model (e.g., regularised linear regression) or a non-parametric model (e.g., k-NN) is more appropriate. Consider bias-variance trade-off, interpretability, and computational constraints, and outline a concrete workflow (pre-processing, model choice, hyper-parameter tuning).

Q0 Hybrid approaches (hard).
Describe a pipeline that combines parametric and non-parametric learners (for example, using a parametric model for dimensionality reduction before applying k-NN). Discuss the potential advantages (e.g., reduced variance, speed) and possible pitfalls (e.g., error propagation, loss of flexibility).

Q0 Evaluation of neighbourhood size (easy).
A validation experiment yields the following mean-squared-error (MSE) values for different k:

$$k = 1, 5, 10, 20, 50 \quad \Longrightarrow \quad \mathrm{MSE} = 1.20,\ 0.85,\ 0.70,\ 0.68,\ 0.72.$$

(Analyse the trend in bias and variance that these numbers suggest.

(Recommend the optimal k and justify your choice; comment on whether a finer grid around the optimum would be useful.

Q0 Theoretical universality (hard).
Explain why, under standard regularity conditions, the k-NN regressor with $k \to 1$ and $N \to \infty$ converges to the conditional expectation $\mathbb{E}[Y \mid \mathbf{X} = \mathbf{x}]$. Sketch the main steps of the proof (law of large numbers, consistency of the nearest-neighbour radius, etc.) without entering full technical detail.